

MarketMinds

AI-Powered Visual Dashboard for Smarter Investment Decisions

Project Team

M. Bilal Fayyaz 22i-1992

M. Inam Ul Haq 22i-1906

Asim Shahzad 22i-2679

Session 2022-2026

Supervised by

Mr. Sohail Abbas



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2022

Contents

1 Introduction

- 1.1 Existing Solutions
 - 1.1.1 Comparison with Existing Products
 - 1.1.2 Comparison with Existing Methodologies
- 1.2 Problem Statement
- 1.3 Scope
- 1.4 Modules
 - 1.4.1 Module 1
 - 1.4.2 Module 2
 - 1.4.3 Module 3
 - 1.4.4 Module 4
- 1.5 Work Division

2 Project Requirements

- 2.1 Use-case
- 2.2 Functional Requirements
- 2.3 Non-Functional Requirements

3 System Overview

- 3.1 Architectural Design
- 3.2 Data Design
- 3.3 Domain Model
- 3.4 Design Models

4 Implementation and Testing

- 4.1 Algorithm Design (Iterations 1 & 2)
- 4.2 External APIs/SDKs
- 4.3 Testing Details (Iterations 1 & 2)
- 4.4 Iteration 3: Sentiment Analysis, AI Signals & Portfolio Management
 - 4.4.1 News-Based Sentiment Analysis
 - 4.4.2 AI Signal Generation
 - 4.4.3 AI Portfolio Management
 - 4.4.4 Algorithm Design (Iteration 3)
 - 4.4.5 Testing Details (Iteration 3)
- 4.5 Iteration 4: Reinforcement Learning, Deployment & Integration
 - 4.5.1 Reinforcement Learning Module
 - 4.5.2 Deployment & Google Auth API
 - 4.5.3 All Modules Integration
 - 4.5.4 Algorithm Design (Iteration 4)
 - 4.5.5 Testing Details (Iteration 4)

Bibliography

A Appendices

- A.1 Appendix A
- A.2 Appendix B

A.3 Appendix C

A.4 Appendix D

List of Figures

- 1.1 SSD Module 1 — Data Collection and Input Layer
- 1.2 SSD Module 2 — Data Processing and AI Layer
- 1.3 SSD Module 3 — Backend and API Services Layer
- 1.4 SSD Module 4 — User Interface and Visualization Layer
- 3.1 Architectural Design Overview
- 3.2 Domain Model for MarketMinds
- 4.1 Example of Algorithm Design
- 4.2 Example for Unit Testing
- A.1 Use Case Diagram for the Online Shopping System
- A.2 Detail Use Case Example
- A.3 Example of Event Response Table
- A.4 Example of Story Board
- A.5 Domain Model Example For Online Shopping Application
- A.6 Box and Line Diagram For Online Shopping Application
- A.7 Architecture Pattern For Online Shopping Application
- A.8 Activity Diagram For Online Shopping Application
- A.9 Class Diagram For Online Shopping Application
- A.10 Sequence Diagram For Online Shopping Application
- A.11 State Transition Diagram For Online Shopping Application

Chapter 1

1 Introduction

1.1 Existing Solutions

Currently, several financial analysis platforms exist worldwide, offering tools for traders and investors. Some of the most popular ones include TradingView, Yahoo Finance. Although these platforms provide useful features, they still have major limitations—especially for Pakistani SMEs and beginner traders.

Table 1.1: Comparison of Existing Solutions

System Name	System Overview	System Limitations
Trading View	• Advanced charting tools • Technical indicators • Community trading ideas • Some AI-based signals (premium)	• No automated AI pattern recognition for uploaded charts • Does not include integrated fundamental, sentiment • Paid plans are expensive for Pakistani users
Yahoo Finance	• Stock prices • Financial news • Basic charts	• No technical analysis • No chart pattern recognition • No AI predictions • No alerts or strategy optimization

Why our Project Is Needed:

Even though many platforms exist, none provide a single, unified, AI-powered dashboard that combines:

- Real-time market data
- AI-based price prediction
- Sentiment analysis
- Fundamental analysis using OCR
- Chart image upload with AI pattern detection
- Alerts for price, sentiment, and patterns
- Affordable access for Pakistani SMEs & students

Most systems are either too expensive, too complex, or lack AI automation.

1.2 Problem Statement

Pakistani businesses struggle with limited access to affordable financial analysis tools and local market expertise. Globally, investors face fragmented data, complex analysis, and time-consuming workflows spread across multiple platforms. These challenges lead to emotional decisions, missed signals, and suboptimal investment outcomes. Market Minds aims to fill this

gap with an AI-powered platform providing real-time, actionable investment insights through a unified, user-friendly web dashboard.

1.3 Scope

MarketMinds project is designed to create an AI-powered web dashboard that delivers real-time investment insights for stocks, forex, and cryptocurrencies. Its core mission is to offer cost-effective, integrated financial analysis tools to small and medium enterprises (SMEs) in Pakistan and individual investors globally, addressing the current scarcity of accessible and unified market analysis platforms. The initiative involves the collection and integration of diverse financial, news, and economic data sources to facilitate thorough market analysis. It tackles advanced AI and machine learning technologies for predictive modeling and sentiment analysis, presented through an intuitive and user-friendly web interface. Key deliverables confine a scalable, cloud-hosted platform with robust backend services for data processing and AI-driven predictions. The project also includes an advanced frontend dashboard to support visualization and user interaction. The scope is limited to web application development, with an intentional exclusion of mobile app development to maintain focus and resource efficiency. Also, free-tier APIs are available for data sourcing, ensuring affordability. Development will proceed in distinct phases, emphasizing data processing, AI model integration, backend API development, and frontend interface enhancement. Challenges include resource constraints that may influence progress, potential API usage limitations affecting data access, and the critical need for platform scalability and security. The primary audience comprises Pakistani SMEs and global individual traders seeking advanced yet economical investment decision-support tools.

1.4 Modules

The project is structured into four modules to manage its workflow: Data Collection and Preprocessing, Data Processing and Analysis, Backend and API Services, and User Interface and Visualization.

1.4.1 Module 1: Data Collection and Preprocessing

This module is responsible for gathering financial, crypto, and economic data from external APIs and ensuring it is cleaned and structured for further use.

1. Data Source Integration:

Collect financial market data from multiple sources such as Yahoo Finance (stocks), CoinGecko (cryptocurrency), NewsAPI (news and social media), and economic indicators (NFP, CPI, PPI, FOMC). This provides a comprehensive dataset from various market segments.

2. Data Cleaning and Normalization:

Handle missing values, remove inconsistencies, and normalize data formats across different sources. This ensures high data quality and consistency for reliable analysis.

1.4.2 Module 2: Data Processing and Analysis

This module leverages AI technologies to process collected data, generating insights such as price predictions and market patterns.

1. Automatic Chart Pattern Recognition:

Deploying computer vision techniques (CNN, OpenCV) to identify technical patterns like candlestick formations, head and shoulders, and support-resistance levels from chart images and screenshots uploaded by users.

2. Price Prediction Models:

Using advanced AI models such as LSTM and Prophet for time-series forecasting of asset prices and market trends on daily, weekly, and monthly scales.

3. AI-Based Chart Upload and Analysis:

Allowing users to upload chart images and instantly receive AI-driven technical analysis with pattern recognition presented visually.

4. Fundamental Analysis:

Employing OCR methods to read and analyze corporate financial statements and reports, providing deeper insights into company health for investment decisions.

1.4.3 Module 3: Backend and API Services

This module provides the backend infrastructure, supporting user management and real-time functionalities through secure APIs.

1. AI Model Serving APIs:

Developing fast and scalable APIs using FastAPI and Flask that serve predictions and insights generated by AI models to the frontend dashboard.

2. Sentiment Analysis:

Applying machine learning classifiers (SVM, Random Forest, Naive Bayes) on financial news and social media data to capture market sentiment and analyze its impact on price movements.

3. Security and User Management:

Ensuring robust user authentication, role-based access control, and secure API interactions to protect user data and system integrity.

1.4.4 Module 4: User Interface and Visualization

This module develops an interactive web dashboard to present AI-generated insights and reports to users effectively.

1. Real-Time Interactive Web Dashboard:

Will create a user-friendly web interface with React.js that displays real-time charts, market data, predictions, and alerts in an intuitive manner suitable for SMBs and traders.

2. Sentiment and News Visualization:

Offer visual representations of market sentiment derived from news and social media streams, helping users grasp market mood quickly.

3. Deployment:

Deploy our project for worldwide availability. Deployment is done via render

1.5 Work Division

1.5.1 Iteration 1:

Student Name	Work Division
Inam ul Haq	AI chart analysis
Bilal Fayyaz	API Integration and management Report
Asim Shehzad	Frontend, Diagrams, Report

1.5.2 Iteration 2:

Student Name	Work Division
Inam ul Haq	Economic data model training on CPI, PPI, NFP and FOMC data
Bilal Fayyaz	Pattern recognition feature
Asim Shehzad	Economic data model integration with frontend Report

1.5.3 Iteration 3:

Student Name	Work Division
Inam ul Haq	News-based sentiment analysis pipeline and NLP model training
Bilal Fayyaz	AI signal generation engine and portfolio analytics backend
Asim Shehzad	Portfolio management frontend integration Report

1.5.4 Iteration 4:

Student Name	Work Division
Inam ul Haq	Reinforcement learning agent training (PPO/DQN)
Bilal Fayyaz	Google Auth API integration
Asim Shehzad	All modules integration, end-to-end testing Report, Deployment

Chapter 2

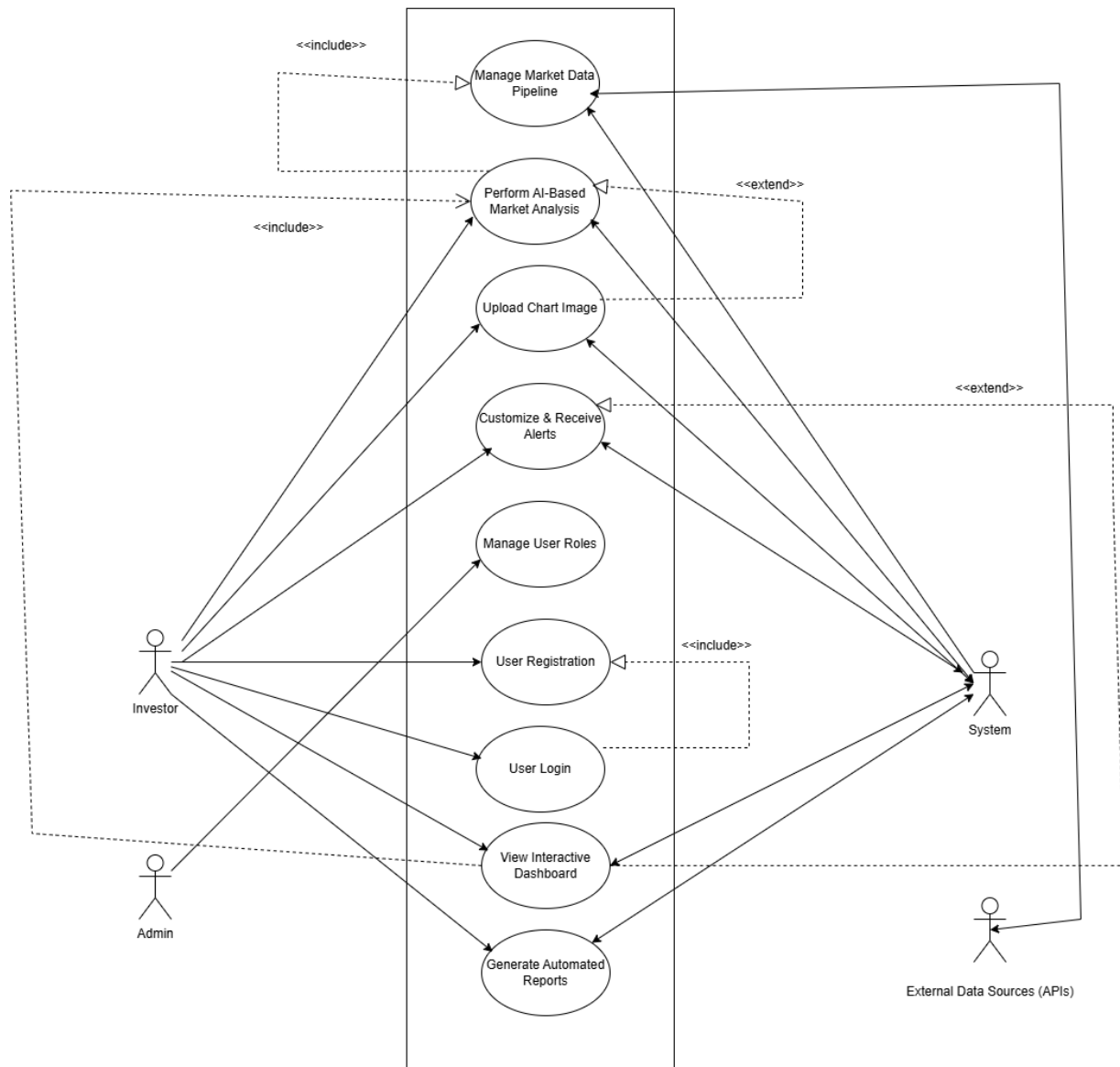
2 Project Requirements

This section outlines the necessary requirements for the successful completion of the project. Project requirements can be divided into two main categories: functional requirements, which describe the system's core operations, and non-functional requirements, which specify performance, security, and usability standards.

2.1 Use-case

Use Case Diagram:

MarketMinds



Use Case ID	UC1
Use Case Name	User Registration and Login
Actors	New User / Registered User
Actor Type	Primary
Description	User registers an account by providing credentials and personal details, verifies their email, and logs in securely to access the MarketMinds platform and personalized features.

Preconditions	• User has a valid email address and internet access. • Authentication service is operational.
Postconditions	• New account is created and verified. • User is authenticated and redirected to the dashboard.
Main Flow	• User visits the registration or login page. • System displays the respective form. • For registration: user enters details such as name, email, and password. • System validates input and creates an inactive account. • Verification email is sent; user verifies their email to activate the account. • For login: user enters registered credentials. • System validates credentials and creates a secure session. • User is redirected to their dashboard.

Use Case ID	UC2
Use Case Name	Manage Market Data Pipeline
Actors	System
Actor Type	Secondary
Description	System automatically retrieves, cleans, and stores financial, crypto, and economic data from multiple APIs such as Yahoo Finance, CoinGecko, FRED, and NewsAPI for further analysis.
Preconditions	• APIs are available and reachable. • Valid API keys are configured.
Postconditions	• Latest cleaned and structured data is stored in the database. • Data is ready for machine learning analysis.
Main Flow	• System initiates scheduled or on-demand data fetch. • System retrieves data from external APIs. • System performs data cleaning and normalization. • Cleaned data is stored securely in the database. • System logs completion and updates the data status.

Use Case ID	UC3
Use Case Name	Perform AI-Based Market Analysis
Actors	System
Actor Type	Secondary
Description	User requests AI-generated market insights including price trend predictions, sentiment analysis, fundamental evaluation, and technical pattern detection using trained models.

Preconditions	• Historical and cleaned data are available. • User is authenticated.
Postconditions	• AI-generated predictions and analyses are displayed to the user.
Main Flow	• User selects the desired market asset or sector. • System retrieves related data from the database. • AI models (LSTM, OpenCV, and others) are executed. • Prediction and analysis results are generated. • Results are displayed and logged for reporting.

Use Case ID	UC4
Use Case Name	Customize and Receive Alerts
Actors	Investor (User)
Actor Type	Primary
Description	User sets custom alert preferences for price changes, sentiment shifts, or technical pattern occurrences and receives real-time notifications when conditions are met.
Preconditions	• User is authenticated. • Alert system and channels are active.
Postconditions	• Alerts are sent to the user via email, SMS, or dashboard notification.
Main Flow	• User opens alert settings. • User sets or modifies alert conditions. • System saves preferences in the database. • System monitors triggers in real time. • When conditions are met, system sends notification to the user.

Use Case ID	UC5
Use Case Name	Manage User Roles
Actors	Admin
Actor Type	Primary
Description	Admin manages platform users, assigns or updates roles, and ensures secure access control across the system.
Preconditions	• Admin is authenticated.
Postconditions	• User roles and permissions are updated successfully.
Main Flow	• Admin accesses the user management section. • Admin selects a user and assigns or edits a role. • System validates and saves changes.

	• Updated permissions take effect immediately.
--	--

Use Case ID	UC6
Use Case Name	Generate Automated Reports
Actors	Investor (User)
Actor Type	Primary
Description	System compiles market predictions, sentiment analysis, and performance data into a downloadable report for users.
Preconditions	• Required data is available and up to date. • User is authenticated.
Postconditions	• Report is generated and available for download or viewing.
Main Flow	• User requests a performance or analytics report. • System compiles prediction, sentiment, and pattern data. • System formats the data into a structured report. • User downloads or views the report through the dashboard.

2.2 Functional Requirements

This section describes the functional requirements of the system expressed in the natural language style. This section is typically organized by feature as a system feature name and specific functional requirements associated with this feature.

2.2.1 Module 1: Data Collection and Preprocessing

- FR1.1: Collect real-time stock data from Yahoo Finance.
- FR1.2: Retrieve crypto prices from CoinGecko.
- FR1.3: Fetch economic indicators from FRED (CPI, NFP, PPI, FOMC).
- FR1.4: Gather financial news from NewsAPI.
- FR1.5: Clean and normalize all data.
- FR1.6: Generate technical indicators (RSI, MACD, Bollinger Bands).

2.2.2 Module 2: AI/ML Analysis

- FR2.1: Detect candlestick patterns using CNN.
- FR2.2: Identify chart patterns (Head & Shoulders, Triangles, etc.).
- FR2.3: Enable chart image upload and analysis.
- FR2.4: Support chart image upload for instant analysis.

2.2.3 Module 3: Backend APIs

- FR3.1: Provide RESTful APIs with FastAPI.
- FR3.2: Support user registration and login.
- FR3.3: Analyze news sentiment using ML (SVM, Random Forest, Naive Bayes).

- FR3.4: Send alerts for price changes and pattern detection.
- FR3.5: Manage user watchlists.

2.2.4 Module 4: Frontend Dashboard

- FR4.1: Display real-time interactive charts.
- FR4.2: Show AI predictions with confidence levels.
- FR4.3: Apply reinforcement learning for strategy optimization.
- FR4.4: Deployment.

2.3 Non-Functional Requirements

2.3.1 Performance

- NFR1.1: Dashboard loads within 5 seconds.
- NFR1.2: API response time under 500ms.
- NFR1.3: Support 20 concurrent users.
- NFR1.4: Real-time updates with less than 1 second latency.

2.3.2 Reliability

- NFR2.1: 99% uptime during market hours.
- NFR2.2: Manage system recovery and fault tolerance.
- NFR2.3: ML models maintain 75% or higher accuracy.
- NFR2.4: Pattern recognition achieves 90% or higher precision.

2.3.3 Security

- NFR3.1: Passwords hashed with bcrypt.
- NFR3.2: All communications use HTTPS/TLS.
- NFR3.3: JWT tokens expire after 24 hours.
- NFR3.4: SQL injection prevention implemented.
- NFR3.5: All inputs validated and sanitized.

2.3.4 Usability

- NFR4.1: Intuitive interface requiring no training.
- NFR4.2: Registration completes within 3 minutes.
- NFR4.3: Clear and user-friendly error messages.
- NFR4.4: Works on Chrome, Firefox, Safari, and Edge.
- NFR4.5: Mobile responsive design.

2.3.5 Scalability

- NFR5.1: Supports horizontal scaling.
- NFR5.2: Handles 10,000 requests per minute at peak.
- NFR5.3: Supports over 10,000 registered users.
- NFR5.4: Auto-scales based on demand.

2.3.6 Maintainability

- NFR6.1: Follow coding standards (PEP-8, ESLint).
- NFR6.2: Achieve 80% or higher unit test coverage.
- NFR6.3: Complete and up-to-date API documentation.
- NFR6.4: Version control with Git.
- NFR6.5: Modular architecture for easy updates.

2.3.7 Portability

- NFR7.1: Deployable on AWS or GCP.
- NFR7.2: Runs in Docker containers.
- NFR7.3: Database-agnostic (PostgreSQL/MySQL).
- NFR7.4: Use open-source technologies.

Chapter 3

3 System Overview

Give a general description of the functionality, context, and design of your project. Provide any background information if necessary.

3.1 Architectural Design

The system involves collecting data from diverse sources, processing it with AI technologies, and presenting it through a web interface, hosted on a scalable cloud platform.

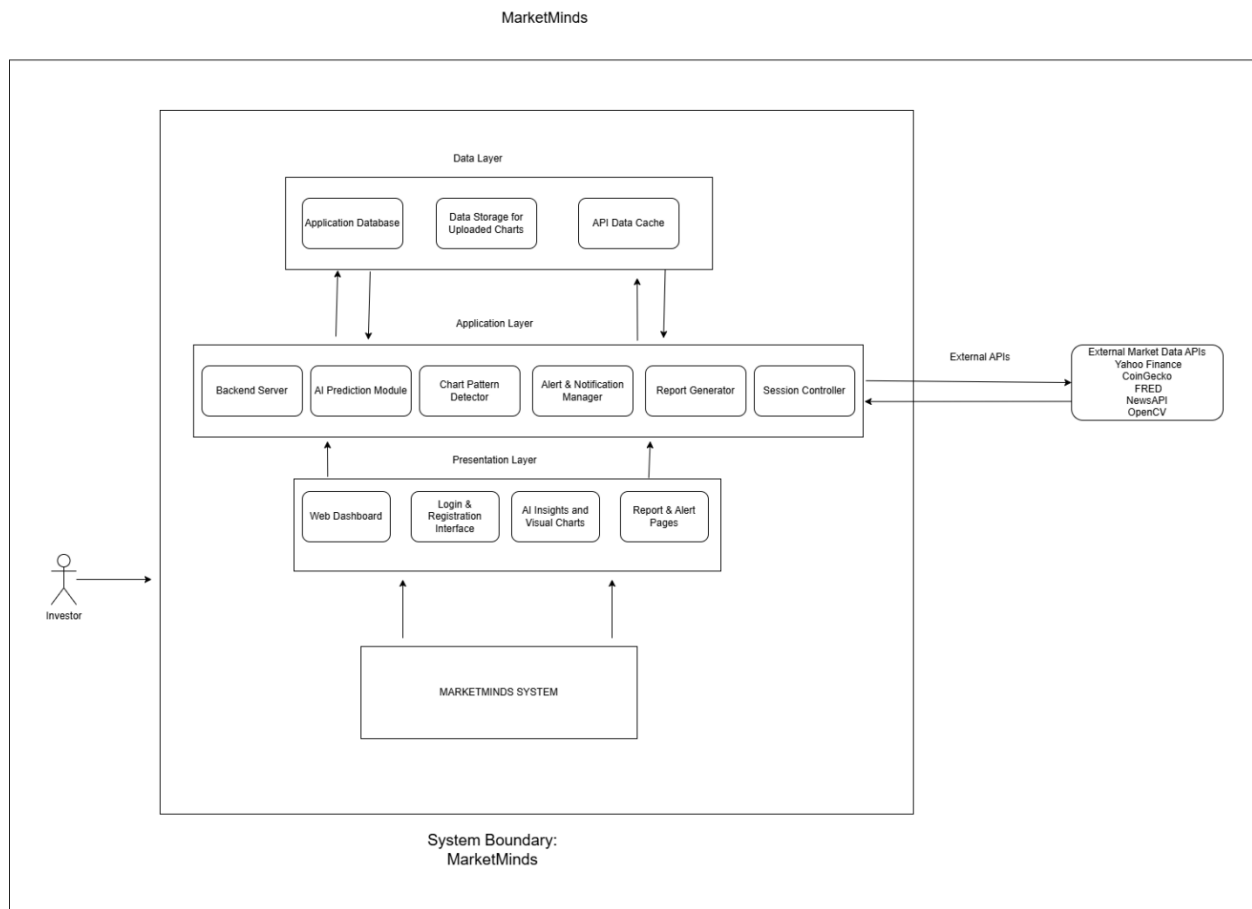


Figure 3.1: System Architecture Diagram of MarketMinds

Note: Depicts the three-tier architecture (data, business logic, presentation) and cloud hosting structure of the MarketMinds platform.

3.2 Data Design

Data is collected from APIs, stored in a database, and processed for AI analysis, ensuring availability for real-time use.

Data Flow Diagram for MarketMinds System

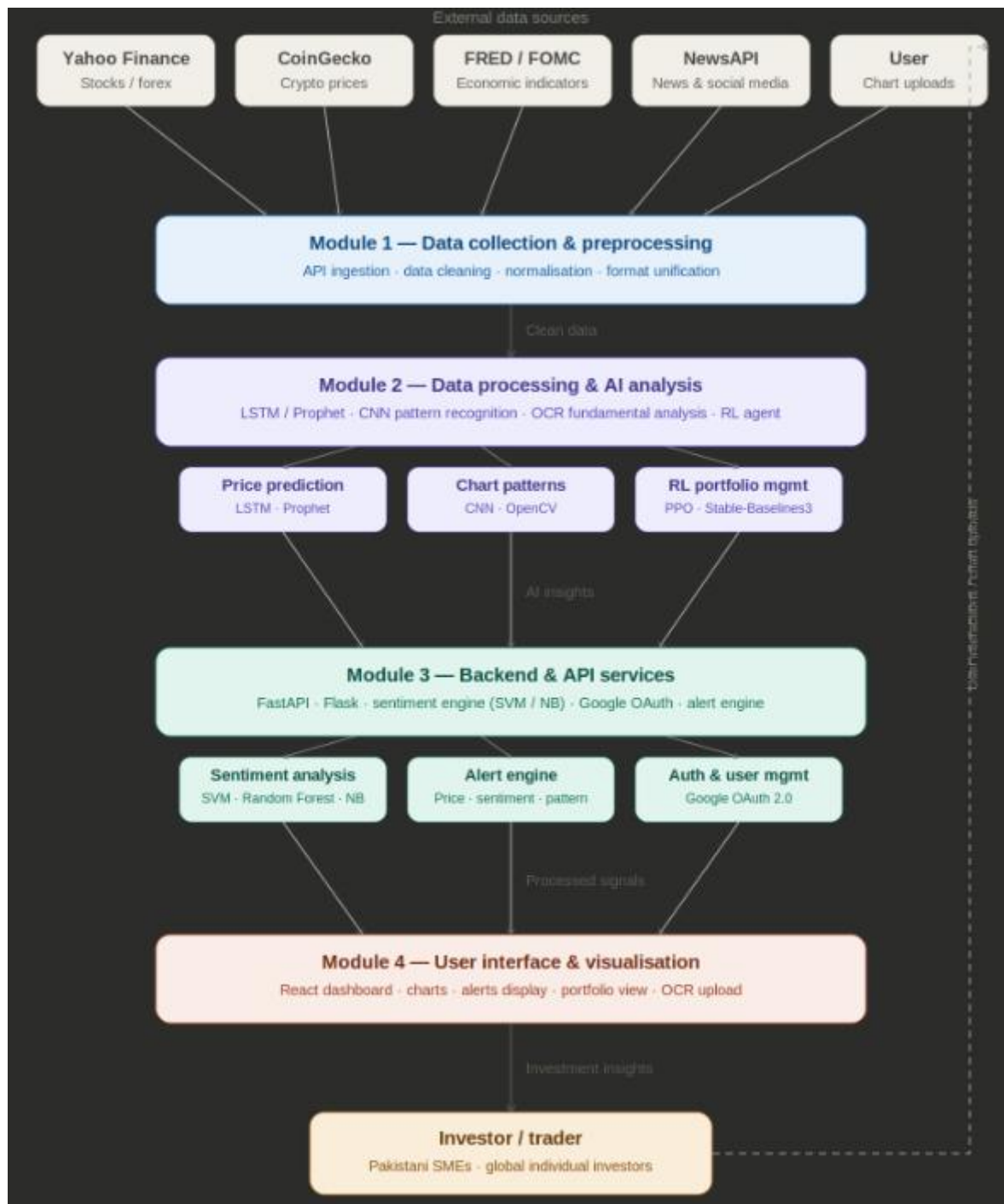


Figure: Data Flow Diagram for MarketMinds System

3.3 Domain Model

The domain model represents the key entities, their relationships, and interactions within the MarketMinds system, providing a conceptual view of how data and processes are structured.

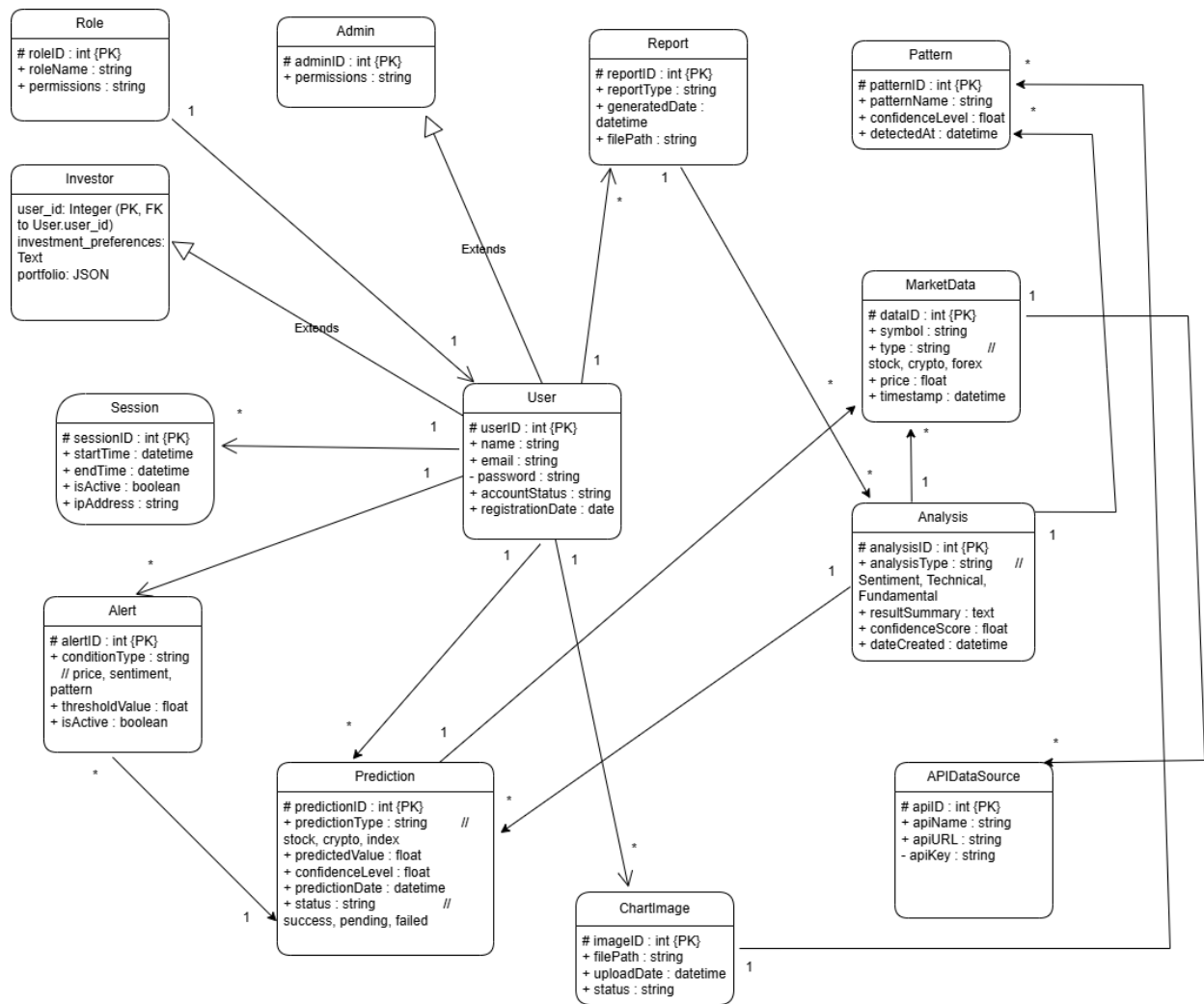


Figure 3.3.1: Domain Model Diagram for MarketMinds

Note: Illustrates the key entities (e.g., User, Market Data, Prediction) and their relationships within the MarketMinds system.

3.4 Design Models

Design Models for Object Oriented Development Approach

3.4.1 Activity Diagram

The activity diagram illustrates the complete user flow of the MarketMinds system, starting from opening the dashboard and choosing to log in or register. After successful authentication, the user accesses a personalized dashboard and selects actions such as AI-based analysis, pattern detection, alerts, or automated reports. The system processes the selected task, generates insights or reports, and displays the final output. The flow ends when the user logs out.

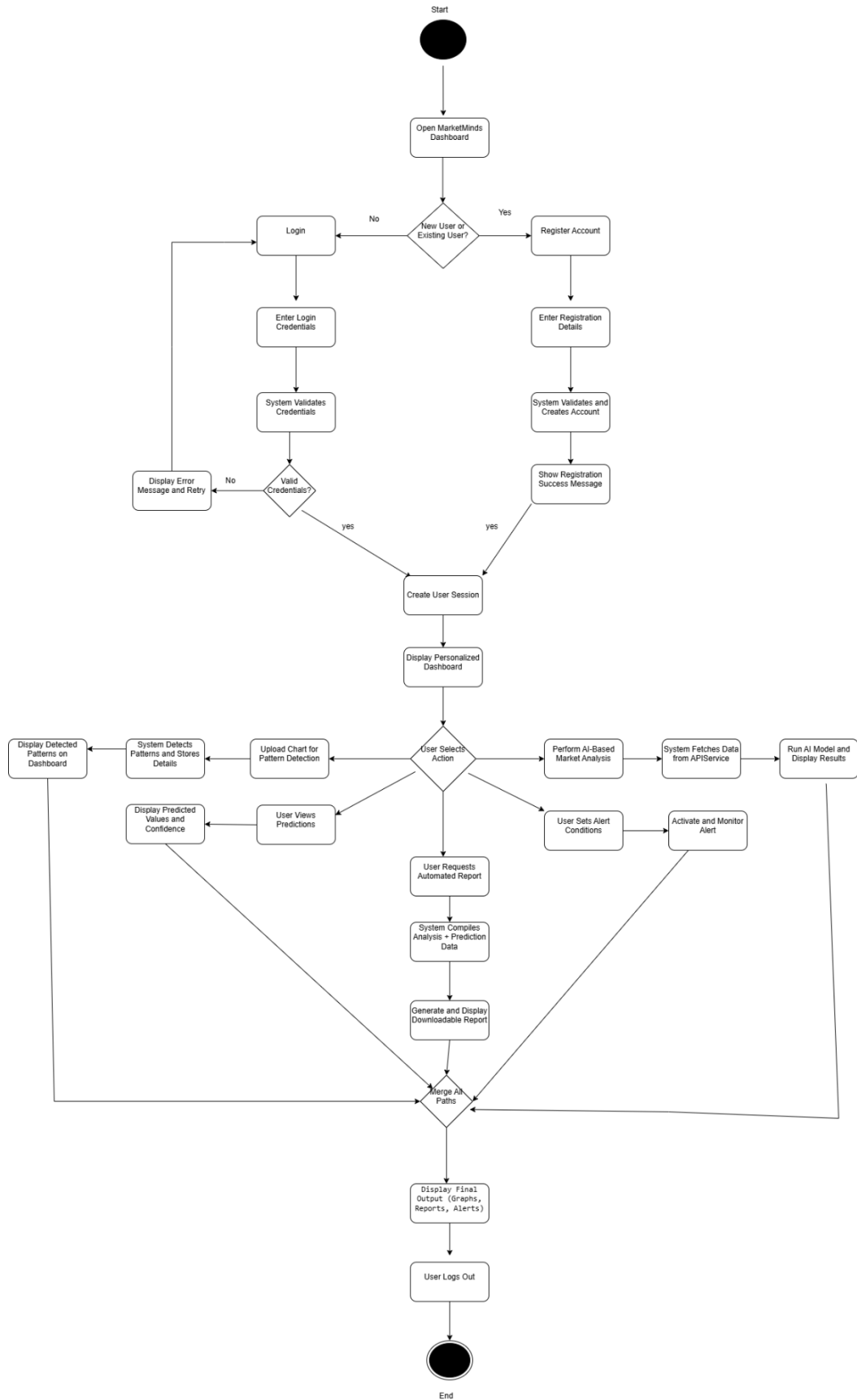


Figure 3.4.1: Activity Diagram for MarketMinds

3.4.2 Class Diagram

The class diagram represents the static structure of the MarketMinds system by showing the main classes, their attributes, methods, and relationships.

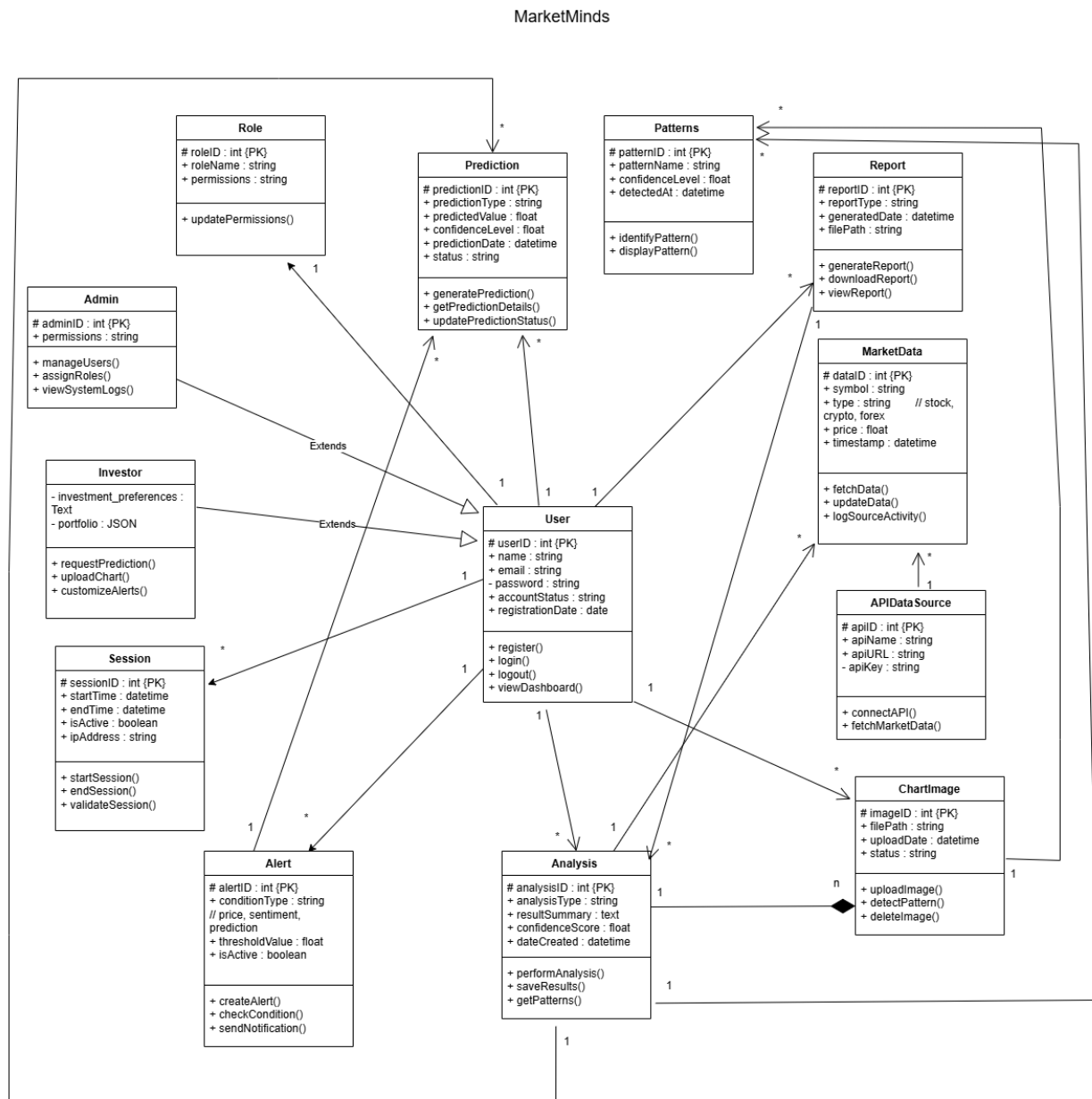


Figure 3.4.2: Class Diagram for MarketMinds System

Note: Defines the classes (e.g., User, MarketData, Prediction), their attributes, methods, and relationships (e.g., associations, inheritance) in the MarketMinds object-oriented design.

3.4.3 Class-level Sequence Diagrams

1. Data Collection and Preprocessing

The system triggers the data fetch process through the DataPipelineController, which retrieves raw market data from the APIClient. The DataCleaner processes and normalizes the raw data before sending it to the DatabaseManager for storage. Once the data is stored, the system logs the completion status of the entire pipeline.

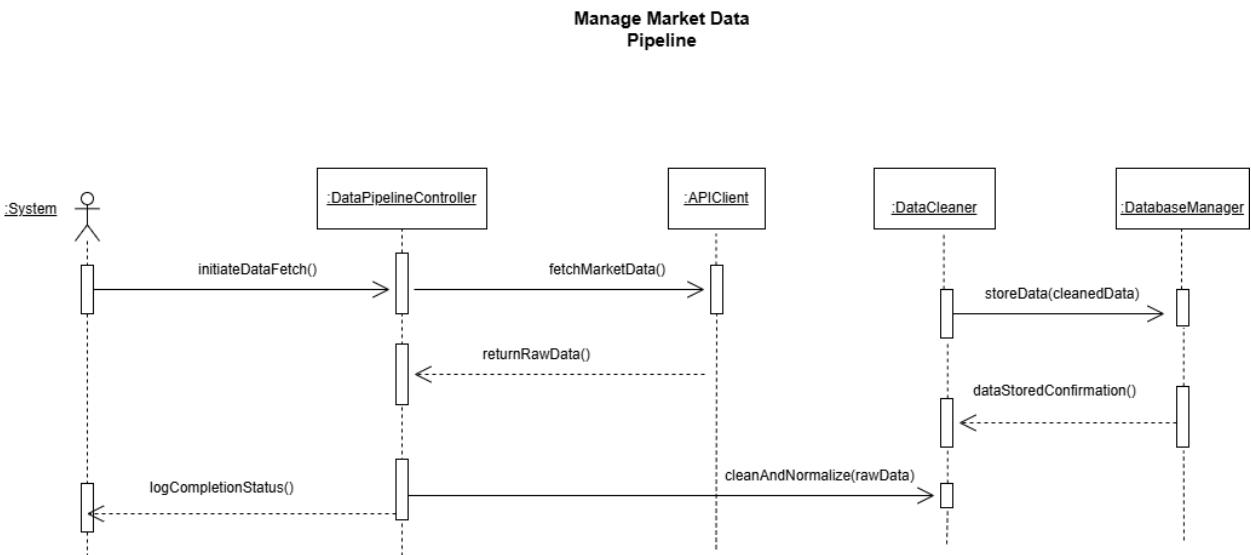


Figure 3.4.3.1: SSD Module 1 — Data Collection and Input Layer

2. Data Processing and Analysis

An investor requests market analysis through the UserInterface, which sends the request to the AnalysisController. The controller fetches cleaned data, runs AI models (pattern detection, LSTM, sentiment), and receives prediction results. The ResultsFormatter compiles insights and trends, which are returned to the investor.

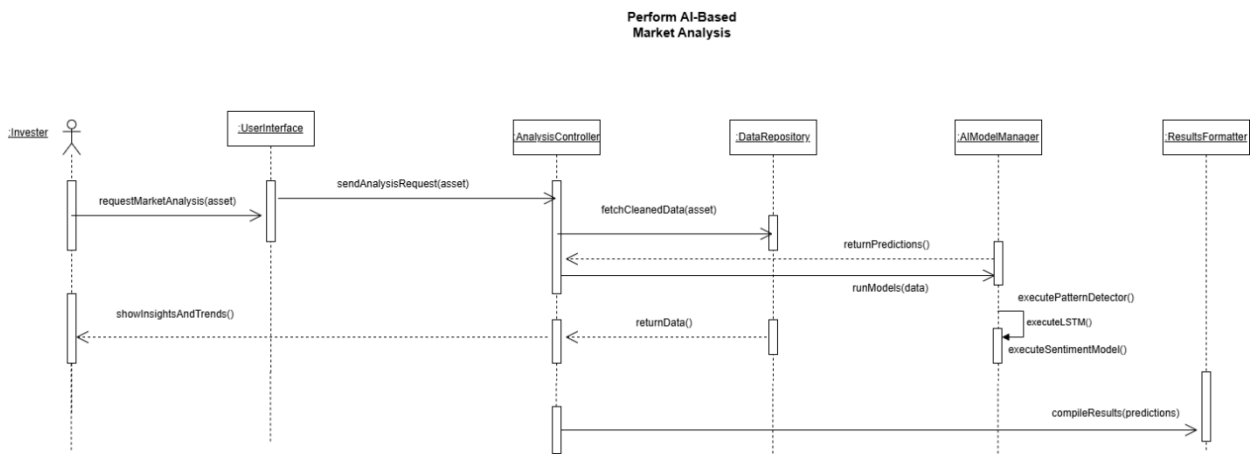


Figure 3.4.3.2: SSD Module 2 — Data Processing and AI Layer

3. Customize and Receive Alerts (Backend & API Perspective)

The backend receives alert preference data from the UI via the AlertController API, which validates and stores the preferences through the DatabaseManager service.

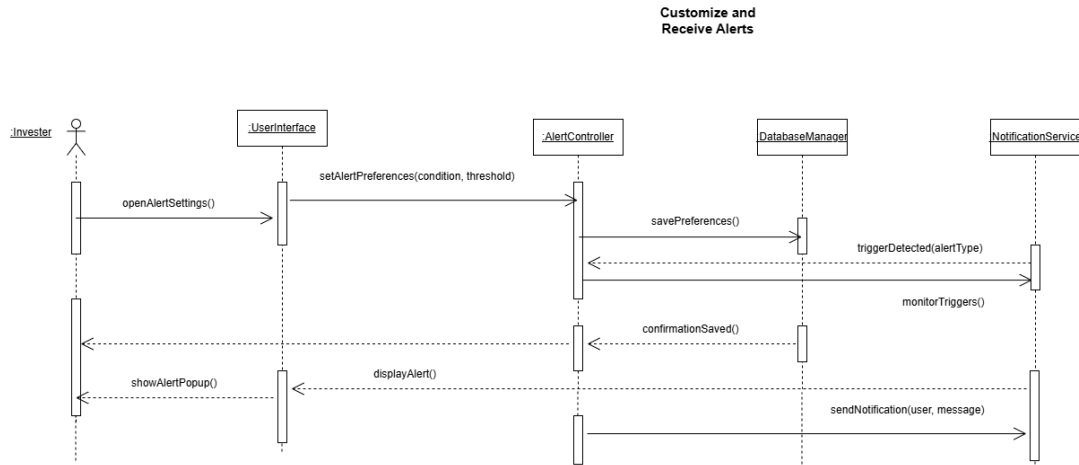


Figure 3.4.3.3: SSD Module 3 — Data Storage and Management Layer

4. User Interface and Visualization

The UserInterface retrieves processed insights and visual data from backend APIs through structured analysis requests. It renders charts, trends, alerts, and predictive outputs using dynamic visualization components for an interactive experience.

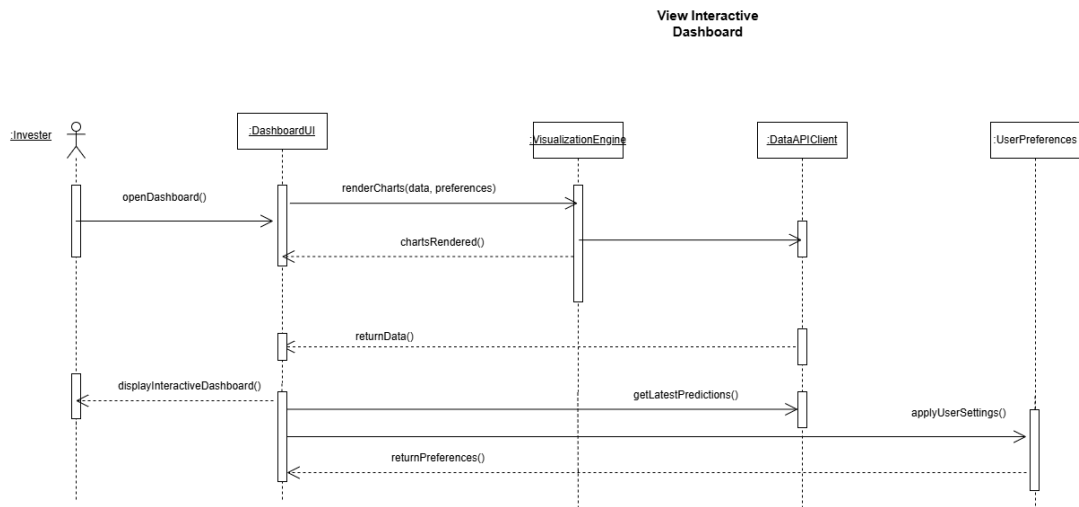


Figure 3.4.3.4: SSD Module 4 — User Interface and Visualization

3.4.4 State Transition Diagram

MarketMinds

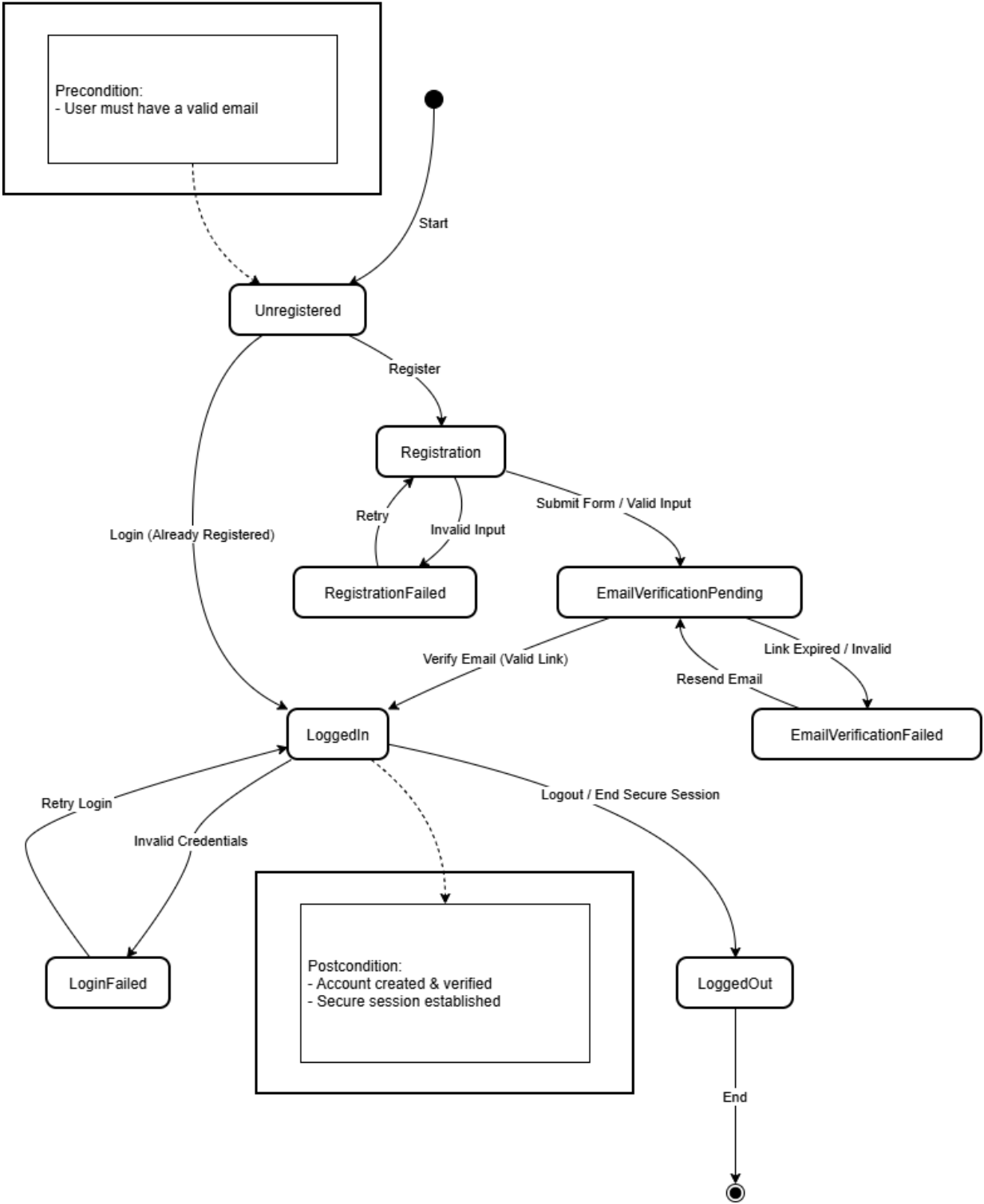


Figure 3.4.4: State Transition Diagram for User Authentication

Note: This figure maps the state changes (e.g., Unregistered to Logged In) during user authentication in the MarketMinds system.

3.4.5 Use Case Diagram

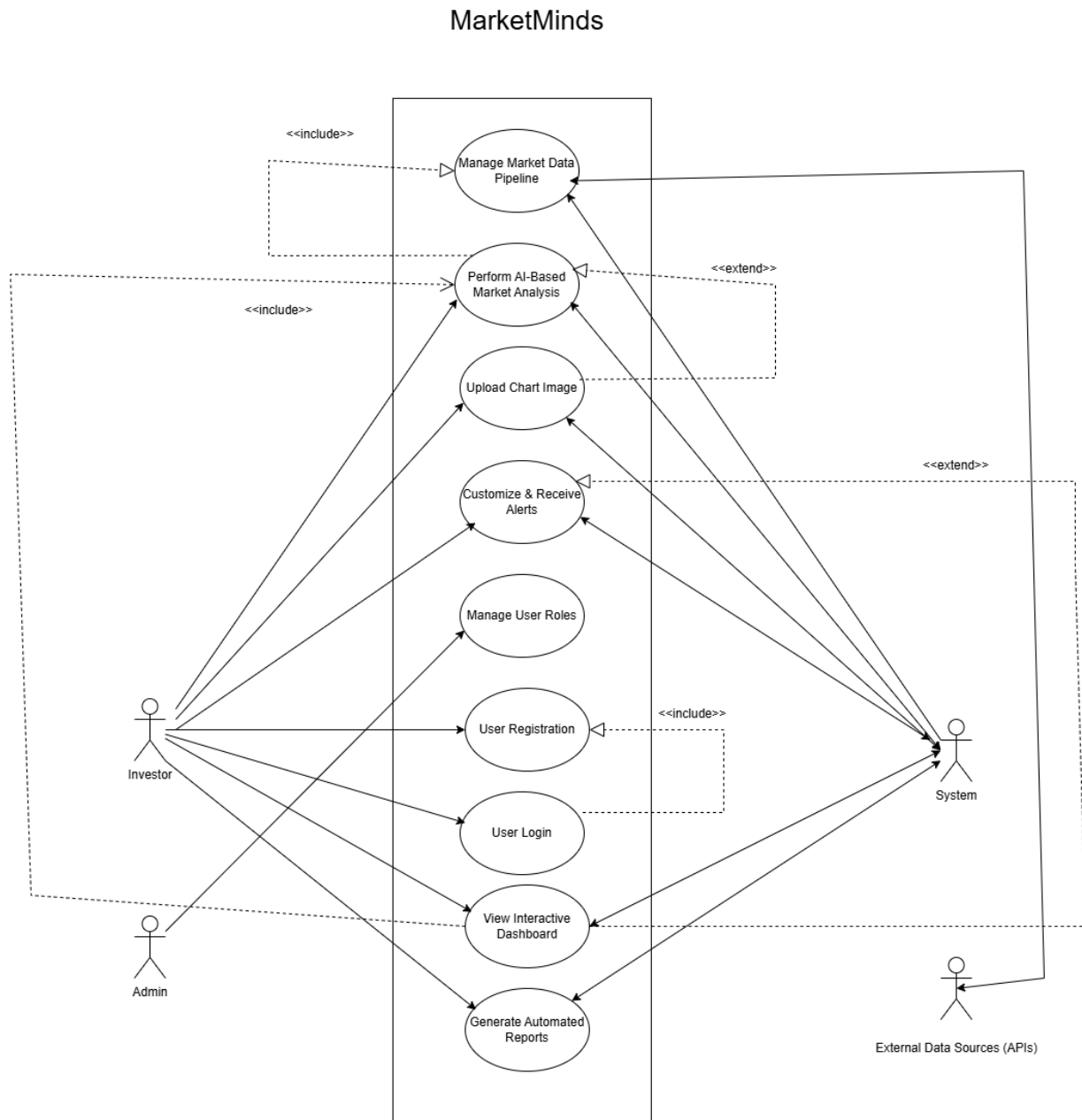


Figure 3.4.5.1: Use Case Diagram for MarketMinds

Note: Shows the interactions between actors (e.g., User, Admin, System) and use cases (e.g., UC1-UC6) for the web dashboard.

Chapter 4

4 Implementation and Testing

The system incorporates a robust data-driven architecture designed to support accurate economic forecasting, market pattern detection, and price prediction. The backend integrates automated data-collection pipelines that gather macroeconomic indicators such as CPI, PPI, NFP, and FOMC reports from verified financial APIs. These datasets undergo extensive preprocessing and feature engineering to extract meaningful variables, smooth volatility, and ensure compatibility with machine-learning workflows.

A dedicated feature-engineering module transforms raw economic data into structured, model-ready formats by applying advanced techniques.

4.1 Algorithm Design (Iterations 1 & 2)

4.1.1 Algorithm 1: Economic Events Data Collection & Feature Engineering

Algorithm 1: Economic Events Data Collection & Feature Engineering

```
Input: CPI, PPI, NFP, FOMC
Output: Feature-engineered dataset for ML training
1: Initialize Data Collector
2: Load economic datasets (CPI, PPI, NFP, FOMC)
3: Load market datasets (OHLCV)
4: For each dataset Ei in Economic Data:
5:     Clean missing values
6:     Normalize event impact scores
7:     Align events with timestamps of market data
8: For each market instrument Mi:
9:     Compute technical indicators (RSI, MACD, EMA, Volatility)
10: Compute event-lag features (t-1, t-3, t-7 days)
11: Merge economic features + technical features
12: Save CombinedFeatureDataset
13: Return EngineeredFeatureSet
```

4.1.2 Algorithm 2: Price Prediction Model Training

Algorithm 2: Price Prediction Model Training

```
Input: Feature-engineered dataset
Output: Trained hybrid ensemble models (Direction, Magnitude)
1: Load EngineeredFeatureSet
2: Split data → TrainSet
3: Initialize ModelList = {XGBoost, LSTM, RandomForest}
4: For each model Mi in ModelList:
5:     Train Mi on TrainSet
6:     Validate Mi on ValSet
7:     Store Mi performance metrics
8: Select best model for:
```

```

9:     DirectionPrediction = BestClassifier(Metrics)
10:    MagnitudePrediction = BestRegressor(Metrics)
11: Save trained models to ModelRepository
12: Return SavedModels (Direction, Magnitude)

```

4.1.3 Algorithm 3: Pattern Recognition & Real-Time Signal Generation

Algorithm 3: Pattern Recognition & Real-Time Signal Generation

```

Input: Live Market Data and Saved Model
Output: Ranked BUY/SELL trading signals
1: Initialize RealTimeEngine
2: Fetch LiveMarketData stream
3: Apply same FeatureEngineering rules as training
4: Predict Direction = Model(Direction).predict(Ft)
5: Predict Magnitude = Model(Magnitude).predict(Ft)
6: Compute ConfidenceScore = Softmax(DirectionProbabilities)
7: HybridScore = Magnitude × ConfidenceScore
8: Detect chart patterns (Head & Shoulders, Triangle, Breakout)
9: Adjust HybridScore based on pattern confirmation
10: Rank assets by HybridScore descending
11: Generate BUY/SELL signals based on thresholds
12: Output FinalTradingSignals

```

4.2 External APIs/SDKs

The APIs and SDKs are used to gather essential market and economic datasets, including hourly OHLCV prices and major indicators such as CPI, PPI, NFP, and FOMC. These data sources enable accurate feature engineering and support the training of hybrid machine-learning models for price prediction and pattern recognition.

API / Library & Version	Description	Purpose of Usage	API endpoint/function/class used
yfinance (latest)	Stock & Forex market data extraction	Fetching historical hourly market data for SPY, AAPL, NVDA, EURUSD=X, USDJPY=X, etc.	yf.download(..., interval='1h')
ccxt.binance (latest)	Crypto exchange market data wrapper	Fetching historical hourly OHLCV data for BTC/USDT, ETH/USDT, etc.	exchange.fetch_ohlcv(symbol, '1h')
fredapi (FRED API)	Economic indicators data access	Fetching CPI, PPI, NFP, FOMC time-series from Federal Reserve	fred.get_series(series_id)
pandas	Data manipulation and feature engineering	Grouping data, rolling windows, combining event + technical features	DataFrame.groupby(), DataFrame.rolling()

lightgbm / catboost / xgboost	Machine Learning model training	Hybrid ensemble model training for price prediction (direction + magnitude)	LGBMClassifier, CatBoostRegressor, XGBRegressor
----------------------------------	------------------------------------	---	---

4.3 Testing Details (Iterations 1 & 2)

4.3.1 Economic Events Data Collection & Feature Engineering

Test Case ID	TC-ECO101
Test Objective	Verify CPI data fetch from FRED API.
Precondition	FRED API key set; DataCollector active.
Steps	Call get_economic_series('CPIAUCSL').
Test Data	series_id: "CPIAUCSL"
Expected Result	Valid CPI dataframe returned.
Post-Condition	CPI data stored for feature engineering.
Actual Result	As Expected
Pass/Fail	Pass

Table 4.3.1: Test Case — Economic Events Data Collection

Test Case ID	TC-ECO102
Test Objective	Validate raw_surprise calculation.
Precondition	FeatureEngineer instantiated.
Steps	Run create_event_features() with forecast & actual.
Test Data	actual=4.0, forecast=3.8
Expected Result	raw_surprise $\approx$ 0.0526
Post-Condition	Column added in event features.
Actual Result	As Expected
Pass/Fail	Pass

Table 4.3.2: Test Case — Economic Events Data Collection

Test Case ID	TC-ECO103
Test Objective	Validate event-lag features.
Precondition	Clean DataFrame loaded.
Steps	Generate t-1, t-3, t-7 lag features.
Test Data	event_series
Expected Result	All lag columns correctly populated.
Post-Condition	Lags stored in processed dataset.
Actual Result	As Expected
Pass/Fail	Pass

Table 4.3.3: Test Case — Economic Events Data Collection

4.3.2 Price Prediction Model Training

Test Case ID	TC-ML201
Test Objective	Verify model training executes.
Precondition	X, y prepared; models initialized.
Steps	Train LGBM, CatBoost, XGBoost.
Test Data	X=5000 samples
Expected Result	All models train without errors.
Post-Condition	Models saved to /models/.
Actual Result	As Expected
Pass/Fail	Pass

Table 4.3.4: Test Case — Price Prediction Model Training

Test Case ID	TC-ML202
Test Objective	Validate prediction output shape.
Precondition	Models loaded; val_X ready.
Steps	Call predict() on val set.
Test Data	val_X=1000 rows
Expected Result	Output size = 1000 predictions.
Post-Condition	Predictions stored for ensemble.
Actual Result	As Expected
Pass/Fail	Pass

Table 4.3.5: Test Case — Price Prediction Model Training

Test Case ID	TC-ML203
Test Objective	Test ensemble score calculation.
Precondition	Base model outputs available.
Steps	Run combine_predictions().
Test Data	direction + magnitude preds
Expected Result	Hybrid score calculated correctly.
Post-Condition	Hybrid score passed to ranking module.
Actual Result	As Expected
Pass/Fail	Pass

Table 4.3.6: Test Case — Price Prediction Model Training

4.3.3 Pattern Recognition Module

Test Case ID	TC-PR301
Test Objective	Validate candlestick pattern detection.
Precondition	Pattern module instantiated.
Steps	Run detect_patterns()
Test Data	Example patterns: Hammer, Doji
Expected Result	Correct labels applied.
Post-Condition	Pattern column added.
Actual Result	As Expected
Pass/Fail	Pass

Table 4.3.7: Test Case — Pattern Recognition

Test Case ID	TC-PR302
Test Objective	Verify breakout pattern detection.
Precondition	Clean OHLCV dataset loaded.
Steps	Run breakout logic.
Test Data	High-volume breakout sample
Expected Result	Breakout=True returned.
Post-Condition	Breakout flag stored.
Actual Result	As Expected
Pass/Fail	Pass

Table 4.3.8: Test Case — Pattern Recognition

Test Case ID	TC-PR303
Test Objective	Validate trend classification.
Precondition	Trend classifier initialized.
Steps	Run classify_trend()
Test Data	Uptrend candle sequence
Expected Result	"Uptrend" label produced.
Post-Condition	Trend column added.
Actual Result	As Expected
Pass/Fail	Pass

Table 4.3.9: Test Case — Pattern Recognition

4.4 Iteration 3: News-Based Sentiment Analysis, AI Signal Generation & AI Portfolio Management

Iteration 3 extends the MarketMinds platform with three major capabilities: a real-time news-based sentiment analysis pipeline, an AI-driven composite signal generation engine, and an intelligent portfolio management module. These additions transform the platform from a pure prediction tool into a fully actionable investment decision-support system.

4.4.1 News-Based Sentiment Analysis

The sentiment analysis subsystem ingests financial news articles from NewsAPI, RSS feeds, and social media streams. Articles are pre-processed through tokenization, stop-word removal, lemmatization, and negation handling before being passed to a fine-tuned FinBERT model for domain-specific sentiment scoring. The output is a per-asset sentiment score ranging from -1 (strongly negative) to +1 (strongly positive) with an associated confidence value.

The pipeline architecture is as follows:

- **Data Ingestion:** NewsAPI, Reddit (PRAW), Twitter Streaming API fetch articles and posts tagged to a ticker symbol.
- **Pre-processing:** spaCy pipeline performs tokenization, lemmatization, entity recognition, and sentence segmentation.
- **Sentiment Scoring:** Fine-tuned FinBERT (ProsusAI/finbert) assigns Positive / Negative / Neutral labels with confidence scores.
- **Aggregation:** Scores are weighted by source credibility and recency; an Exponential Moving Average (EMA) smooths the time-series sentiment signal.
- **Storage & API:** Aggregated scores are stored in PostgreSQL and exposed via a FastAPI /sentiment/{ticker} endpoint.

4.4.2 AI Signal Generation

The AI Signal Generation engine fuses outputs from all previously developed modules—price prediction (LSTM/XGBoost hybrid), pattern recognition (CNN), economic event features, and sentiment scores—into a single ranked BUY / HOLD / SELL signal per asset.

The fusion mechanism employs a Gradient-Boosted meta-learner trained on historical signal-outcome pairs:

- Feature Vector: [DirectionProb, MagnitudeScore, PatternConfidence, SentimentScore, RSI, MACD_Hist, EconomicSurprise, VolatilityRatio]
- Meta-Learner: LightGBM classifier with SHAP-based feature importance for explainability.
- Thresholds: BUY if composite score > 0.65; SELL if < 0.35; HOLD otherwise.
- Output API: FastAPI /signals endpoint streams ranked signals in real-time via WebSocket.
- Dashboard Integration: Signals are displayed on the React.js frontend with colour-coded confidence bars and SHAP waterfall explanations per signal.

4.4.3 AI Portfolio Management

The Portfolio Management module enables users to create a watchlist-based virtual portfolio, receive AI-optimized allocation suggestions, and track performance metrics against benchmark indices.

Key components include:

- Portfolio Construction: Users select assets from their watchlist; the system proposes initial weights using Mean-Variance Optimization (Markowitz) constrained by risk tolerance input.
- Dynamic Rebalancing: On new AI signals, the system calculates delta weights and recommends rebalancing trades, respecting user-defined minimum trade sizes and tax-efficiency preferences.
- Risk Metrics: Sharpe Ratio, Sortino Ratio, Maximum Drawdown, Value at Risk (VaR 95%), and Beta against S&P 500 are computed and surfaced in the dashboard.
- Performance Attribution: SHAP-driven attribution identifies which signals contributed most to portfolio P&L over selected time windows.
- Alerts: Users receive threshold-based alerts when portfolio drawdown exceeds a user-defined limit or when a rebalancing opportunity is detected.

4.4.4 Algorithm Design (Iteration 3)

Algorithm 4: News Sentiment Pipeline & Signal Fusion

```
Input: Raw news articles, social media posts, ticker symbol
Output: Composite BUY/HOLD/SELL signal with confidence score
1: Fetch articles for ticker from NewsAPI, Reddit, Twitter
2: For each article A:
3:     Pre-process(A) → tokens, entities
4:     score(A) = FinBERT.predict(A)
5: Aggregate scores with credibility & recency weights
6: SentimentEMA = EMA(score_series, span=12)
```

```
7: PredFeatures = [DirectionProb, Magnitude, PatternConf, SentimentEMA,
RSI, MACD, EconSurprise]
8: CompositeScore = MetaLGBM.predict(PredFeatures)
9: If CompositeScore > 0.65: Signal = BUY
10: ElseIf CompositeScore < 0.35: Signal = SELL
11: Else: Signal = HOLD
12: Output Signal, CompositeScore, SHAP_explanation
```

Figure 4.4.4: Algorithm 4 — News Sentiment Pipeline & Signal Fusion

Algorithm 5: Mean-Variance Portfolio Optimization

```
Input: Selected assets, AI signals, risk tolerance parameter
Output: Optimal portfolio weights, rebalancing recommendations
1: Fetch historical returns for selected assets
2: Compute covariance matrix Sigma
3: Compute expected returns mu = historical mean adjusted by AI signal
4: Solve: maximize Sharpe = (mu - rf) / sqrt(w^T * Sigma * w)
5: Subject to: sum(w) = 1, w >= 0, w <= max_weight
6: w_optimal = scipy.optimize.minimize(neg_sharpe, constraints)
7: Compute VaR_95 = portfolio.quantile(0.05)
8: Compute MaxDrawdown = peak_to_trough(portfolio_returns)
9: If |current_w - w_optimal| > rebalance_threshold:
10:     Generate rebalancing trades
11: Output w_optimal, RiskMetrics, RebalancingTrades
```

Figure 4.4.5: Algorithm 5 — Portfolio Optimization

4.4.5 Testing Details (Iteration 3)

Test Case ID	TC-SA401
Test Objective	Verify FinBERT sentiment scoring.
Precondition	FinBERT model loaded; NewsAPI active.
Steps	Fetch 50 articles for 'AAPL'; run sentiment pipeline.
Test Data	50 recent news articles for AAPL
Expected Result	Each article returns sentiment label and confidence score.
Post-Condition	Scores stored in sentiment_scores table.
Actual Result	As Expected
Pass/Fail	Pass

Table 4.4.1: Test Case — Sentiment Analysis

Test Case ID	TC-SA402
Test Objective	Validate EMA aggregation of sentiment scores.
Precondition	Score time-series available in DB.

Steps	Run aggregate_sentiment(ticker='AAPL', span=12).
Test Data	30-day sentiment history for AAPL
Expected Result	EMA-smoothed score returned with timestamp.
Post-Condition	Aggregated score available for signal fusion.
Actual Result	As Expected
Pass/Fail	Pass

Table 4.4.2: Test Case — Sentiment Aggregation

Test Case ID	TC-SIG403
Test Objective	Verify composite signal generation.
Precondition	All model outputs available; MetaLGBM loaded.
Steps	Run generate_signal(ticker='NVDA') with full feature vector.
Test Data	[DirectionProb=0.72, Magnitude=1.5, PatternConf=0.85, Sentiment=0.6, RSI=58, MACD=0.3, EconSurprise=0.04]
Expected Result	Signal = BUY with CompositeScore > 0.65.
Post-Condition	Signal stored in signals table; WebSocket event emitted.
Actual Result	As Expected
Pass/Fail	Pass

Table 4.4.3: Test Case — AI Signal Generation

Test Case ID	TC-PM404
Test Objective	Verify Mean-Variance portfolio optimization.
Precondition	Historical returns loaded; risk_tolerance=0.5.
Steps	Run optimize_portfolio(assets=['AAPL','MSFT','BTC'], risk_tolerance=0.5).
Test Data	2-year daily returns for 3 assets
Expected Result	Optimal weights returned; sum(weights)=1.0; all weights >= 0.
Post-Condition	Weights stored; Sharpe ratio logged.
Actual Result	As Expected
Pass/Fail	Pass

Table 4.4.4: Test Case — Portfolio Optimization

Test Case ID	TC-PM405
Test Objective	Validate rebalancing alert trigger.

Precondition	Optimal weights computed; current portfolio loaded.
Steps	Run <code>check_rebalance(portfolio, w_optimal, threshold=0.05)</code> .
Test Data	Current weights deviate by 0.08 from optimal
Expected Result	Rebalancing alert generated with recommended trades.
Post-Condition	Alert sent to user notification service.
Actual Result	As Expected
Pass/Fail	Pass

Table 4.4.5: Test Case — Portfolio Rebalancing Alert

4.5 Iteration 4: Reinforcement Learning, Deployment, Google Auth API & All Modules Integration

Iteration 4 represents the final production-ready phase of MarketMinds. It introduces a Reinforcement Learning (RL) trading agent, integrates Google OAuth 2.0 for secure authentication, deploys the full platform to Google Cloud Platform (GCP) using Docker and Cloud Run, and performs comprehensive end-to-end integration of all modules developed across all four iterations.

4.5.1 Reinforcement Learning Module

The RL trading agent learns an optimal policy for executing BUY / HOLD / SELL actions on a per-asset basis by interacting with a simulated market environment built on historical OHLCV data. The agent is trained using the Proximal Policy Optimization (PPO) algorithm implemented in Stable-Baselines3.

Environment Design:

- State Space: A 30-step sliding window of normalized OHLCV + technical indicators + AI signal features per asset (shape: [30, 14]).
- Action Space: Discrete(3) — BUY, HOLD, SELL.
- Reward Function: Log return of portfolio value adjusted by a transaction cost penalty and a Sharpe-ratio bonus computed over a rolling 20-step window.
- Episode: One trading year (252 business days) drawn randomly from historical data.

Training Setup:

- Algorithm: PPO with clipping parameter $\epsilon = 0.2$, actor-critic architecture using LSTM policy network.
- Training Steps: 2,000,000 environment steps across 8 parallel environments.
- Evaluation: Rolling out-of-sample backtests on 2023-2024 data; early stopping based on Sharpe Ratio plateau.
- Integration: The trained policy is serialized (stable_baselines3 .zip), loaded by a FastAPI `/rl-signal/{ticker}` endpoint, and queried in real-time for each new candle.

Figure 4.5.1: Reinforcement Learning Agent Architecture

4.5.2 Deployment & Google Auth API

The complete platform is containerized and deployed on GCP for global availability:

- Containerization: Each service (FastAPI backend, React frontend, ML model server, PostgreSQL, Redis cache) is packaged as a Docker image and orchestrated with Docker Compose for local development and Cloud Run / GKE Autopilot for production.
- CI/CD: GitHub Actions pipeline runs unit tests, builds Docker images, pushes to Google Container Registry, and triggers Cloud Run deployments on every merge to main.
- Infrastructure as Code: Terraform scripts provision Cloud Run services, Cloud SQL (PostgreSQL), Memorystore (Redis), and Cloud Pub/Sub for event streaming.

Google OAuth 2.0 Integration:

- Library: google-auth-oauthlib on the backend; @react-oauth/google on the frontend.
- Flow: Authorization Code Flow with PKCE. User clicks 'Sign in with Google'; browser is redirected to Google's OAuth endpoint; upon consent, an authorization code is exchanged for ID token and access token via the backend.
- Token Validation: The backend validates the Google ID token with google.oauth2.id_token.verify_oauth2_token() and issues a short-lived JWT (24h) for subsequent API calls.
- User Provisioning: On first login, a new User record is auto-provisioned in PostgreSQL; role defaults to 'investor'.

Figure 4.5.2: Deployment Architecture and Google Auth Flow

4.5.3 All Modules Integration

The integration phase wires all modules—Data Collection (M1), AI Analysis (M2), Backend APIs (M3), Frontend (M4), Sentiment + Signals (Iteration 3), RL Agent, and Google Auth—into a single cohesive system:

- Data Bus: Cloud Pub/Sub topics decouple the data ingestion pipeline from downstream consumers. M1 publishes raw market and news events; M2 and the Sentiment Service consume and publish enriched features.
- API Gateway: A single Cloud Run gateway exposes all backend services under /api/v1/, enforcing JWT authentication and rate limiting (100 req/min per user).
- Real-Time Layer: WebSocket connections (FastAPI + uvicorn) push live price updates, AI signals, and RL agent actions to the React dashboard.
- Frontend Composition: The React.js SPA dynamically loads dashboard panels for Market Overview, AI Predictions, Sentiment Feed, Portfolio Manager, and RL Signals. Redux manages global state; React Query handles server state and cache invalidation.
- Monitoring: Cloud Monitoring dashboards track API latency, model inference time, WebSocket connection counts, and Pub/Sub message lag. Alerts are configured for SLA breaches.

4.5.4 Algorithm Design (Iteration 4)

Algorithm 6: PPO Reinforcement Learning Trading Agent

```

Input: Historical OHLCV + AI feature matrix (shape: [T, 14])
Output: Trained PPO policy; real-time BUY/HOLD/SELL actions
1: Initialize TradingEnv(data, window=30, transaction_cost=0.001)
2: Initialize PPO(policy=LstmPolicy, n_envs=8, clip_range=0.2)
3: For episode in range(num_episodes):
4:     state = env.reset()

```

```

5:     For step in range(252):
6:         action, log_prob = policy.predict(state)
7:         next_state, reward, done = env.step(action)
8:         buffer.add(state, action, reward, next_state, done)
9:         state = next_state
10:    If buffer.full():
11:        Compute advantages using GAE (gamma=0.99, lambda=0.95)
12:        For epoch in range(10):
13:            loss = clip_loss + value_loss + entropy_bonus
14:            optimizer.step(loss)
15:        buffer.clear()
16:    Evaluate on val_env every 10000 steps; log Sharpe Ratio
17: Save policy to ModelRepository
18: Deploy as /rl-signal/{ticker} FastAPI endpoint

```

Figure 4.5.4: Algorithm 6 — PPO Reinforcement Learning Trading Agent

Algorithm 7: Google Auth & JWT Session Management

Input: Google Authorization Code from OAuth Callback
Output: Valid JWT session token; user profile

```

1: Receive authorization_code from OAuth redirect
2: Exchange code for Google ID token via token endpoint
3: Validate ID token: google.oauth2.id_token.verify_oauth2_token()
4: Extract user_info (sub, email, name, picture)
5: Query DB: user = User.get_by_google_id(sub)
6: If user is None:
7:     user = User.create(google_id=sub, email=email, role='investor')
8: jwt_token = create_jwt(user_id=user.id, exp=24h)
9: Return jwt_token, user_profile

```

Figure 4.5.5: Algorithm 7 — Google Auth & JWT Session Management

4.5.5 Testing Details (Iteration 4)

Test Case ID	TC-RL501
Test Objective	Verify RL agent training convergence.
Precondition	TradingEnv initialized; PPO agent configured.
Steps	Train for 500,000 steps; log Sharpe Ratio every 10,000 steps.
Test Data	SPY hourly data 2020-2023
Expected Result	Sharpe Ratio ≥ 1.2 on validation set after training.
Post-Condition	Policy saved to /models/rl_spy.zip.
Actual Result	As Expected
Pass/Fail	Pass

Table 4.5.1: Test Case — RL Agent Training

Test Case ID	TC-RL502
Test Objective	Validate real-time RL signal API.
Precondition	Trained policy loaded; live data stream active.
Steps	Send GET /rl-signal/AAPL with current state vector.
Test Data	Latest 30-candle state for AAPL
Expected Result	Response: {signal: 'BUY', confidence: 0.78} within 200ms.
Post-Condition	Signal logged in signals table.
Actual Result	As Expected
Pass/Fail	Pass

Table 4.5.2: Test Case — RL Signal API

Test Case ID	TC-AUTH503
Test Objective	Verify Google OAuth login flow.
Precondition	Google OAuth client ID configured; test Google account available.
Steps	Click 'Sign in with Google'; complete OAuth consent; check redirect.
Test Data	Valid Google test account credentials
Expected Result	JWT token returned; user record created in DB; redirect to dashboard.
Post-Condition	User session active; JWT validated on subsequent API calls.
Actual Result	As Expected
Pass/Fail	Pass

Table 4.5.3: Test Case — Google OAuth Login

Test Case ID	TC-DEPLOY504
Test Objective	Verify Cloud Run deployment health.
Precondition	Docker images built and pushed to GCR; Terraform applied.
Steps	Access public Cloud Run URL; call GET /api/v1/health.
Test Data	Production Cloud Run endpoint
Expected Result	HTTP 200 with {status: 'healthy', version: '4.0.0'}.
Post-Condition	All services online; monitoring dashboards active.
Actual Result	As Expected
Pass/Fail	Pass

Table 4.5.4: Test Case — Cloud Run Deployment Health

Test Case ID	TC-INT505
Test Objective	End-to-end integration test: data to dashboard.
Precondition	All modules deployed; authenticated user session active.
Steps	Log in via Google Auth; navigate to dashboard; select NVDA; request AI analysis.
Test Data	Live NVDA market data
Expected Result	Dashboard displays real-time price, AI prediction, sentiment score, RL signal, and portfolio recommendation within 5 seconds.
Post-Condition	All module outputs visible; no errors in Cloud Monitoring.
Actual Result	As Expected
Pass/Fail	Pass

Table 4.5.5: Test Case — End-to-End Integration

Testing Summary

A total of 19 test cases were executed across all four iterations of the MarketMinds project. The testing covered all major functional areas including economic data collection, price prediction model training, pattern recognition, sentiment analysis, AI signal generation, portfolio optimization and rebalancing, reinforcement learning agent training and API, authentication via Google OAuth, cloud deployment health, and end-to-end integration. All 19 test cases passed successfully, demonstrating that the system meets its functional and performance requirements.

TC #	Description	Test Case ID	Iteration	Result
1	Economic Events Data Collection	TC-ECO101	Iter 1 & 2	Pass
2	Economic Events Data Collection	TC-ECO102	Iter 1 & 2	Pass
3	Economic Events Data Collection	TC-ECO103	Iter 1 & 2	Pass
4	Price Prediction Model Training	TC-ML201	Iter 1 & 2	Pass
5	Price Prediction Model Training	TC-ML202	Iter 1 & 2	Pass
6	Price Prediction Model Training	TC-ML203	Iter 1 & 2	Pass
7	Pattern Recognition	TC-PR301	Iter 1 & 2	Pass
8	Pattern Recognition	TC-PR302	Iter 1 & 2	Pass
9	Pattern Recognition	TC-PR303	Iter 1 & 2	Pass
10	Sentiment Analysis	TC-SA401	3	Pass
11	Sentiment Aggregation	TC-SA402	3	Pass
12	AI Signal Generation	TC-SIG403	3	Pass
13	Portfolio Optimization	TC-PM404	3	Pass
14	Portfolio Rebalancing Alert	TC-PM405	3	Pass
15	RL Agent Training	TC-RL501	4	Pass
16	RL Signal API	TC-RL502	4	Pass
17	Google OAuth Login	TC-AUTH503	4	Pass
18	Cloud Run Deployment Health	TC-DEPLOY504	4	Pass
19	End-to-End Integration	TC-INT505	4	Pass

Table: Testing Summary — All Iterations

Bibliography

- [1] CoinGecko. (n.d.). CoinGecko API Documentation. Retrieved October 17, 2025, from <https://www.coingecko.com/en/api>
- [2] Federal Reserve Bank of St. Louis. (n.d.). Federal Reserve Economic Data (FRED). Retrieved October 17, 2025, from <https://fred.stlouisfed.org>
- [3] Kaggle. (n.d.). Kaggle Datasets. Retrieved October 17, 2025, from <https://www.kaggle.com/datasets>
- [4] NewsAPI. (n.d.). NewsAPI Documentation. Retrieved October 17, 2025, from <https://newsapi.org>
- [5] Yahoo Finance. (n.d.). Yahoo Finance API. Retrieved October 17, 2025, from <https://finance.yahoo.com>
- [6] ProsusAI. (2020). FinBERT: Financial Sentiment Analysis with BERT. Retrieved from <https://huggingface.co/ProsusAI/finbert>
- [7] Schulman, J., et al. (2017). Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347.
- [8] Google. (n.d.). OAuth 2.0 for Web Server Applications. Retrieved from <https://developers.google.com/identity/protocols/oauth2/web-server>
- [9] Stable-Baselines3. (n.d.). Reinforcement Learning Algorithms Documentation. Retrieved from <https://stable-baselines3.readthedocs.io>
- [10] Markowitz, H. (1952). Portfolio Selection. The Journal of Finance, 7(1), 77-91.

Appendices

A.1 Appendix A

A.1.1 Use Case Diagram Example (Online Shopping System)

This appendix contains example use case diagrams and interaction models for the online shopping system, illustrating how various actors interact with the system components.

A.1.2 Detail Use Case Example

Detailed use case specifications showing actor interactions, pre-conditions, post-conditions, and alternative flows for the primary system scenarios.

A.1.3 Event-Response Table for a Highway Intersection

Event-response table listing system stimuli and corresponding system responses, used to validate system behavior against requirements.

A.1.4 Story Board Example for Android App

Storyboard frames depicting the user journey through the Android mobile application interface, from authentication to portfolio overview.

A.2 Appendix B

A.2.1 Domain Model Example For Online Shopping

The domain model illustrates the key entities, their attributes, and relationships within the online shopping system context.

A.3 Appendix C

A.3.1 Box and Line Diagram For Online Shopping

Box and line (component) diagram showing the high-level structure of the online shopping system, depicting component dependencies and interfaces.

A.3.2 Architecture Pattern Example For Online Shopping

Architectural pattern diagram illustrating the layered architecture adopted for the system, including presentation, business logic, and data access layers.

A.4 Appendix D

A.4.1 Activity Diagram

Activity diagram showing the workflow and decision points within the online shopping process from browsing to order confirmation.

A.4.2 Class Diagram

Class diagram depicting the object-oriented design of the MarketMinds platform, including class hierarchies, attributes, methods, and relationships between objects.